

CSC 1383 Structured Programming

Final Assignment Report

Medicine Stock and Expiry Management

Student Name: Samia Jahan

Student ID: 26103184

Section: E

Submission Date: 09-09-2026

1. Introduction

The community health centre requires an automated system to manage the regular stock of essential medicines for its patients. Stock quantities fluctuate continuously due to daily consumption and periodic replenishment. Proper management is necessary to avoid both shortages of critical medications and excessive stock that may expire before use.

The objective of the developed C programme is to examine current medicine stocks, calculate estimated coverage, dynamically identify shortage and expiry risks, prioritize medicines requiring management attention, and generate an organized stock-status report.

2. Problem Analysis

2.1 Major Requirements

The developed system needs to perform the following major tasks:

1. Analyze six supplied medicines.
2. Calculate estimated stock coverage.
3. Identify the current stock condition based on various risk factors.
4. Assign a management priority.
5. Allow users to search for medicines.
6. Allow new stock to be received.
7. Allow medicines to be issued to patients.
8. Output a sorted summary report.
9. Store the report in a file.

2.2 Initial Dataset

The program uses the six medicines supplied in the assignment:

Code	Medicine	Stock	Daily Requirement	Minimum Stock	Expiry Days	Essentiality
MED01	Oral Saline	120	35	80	45	3
MED02	Paracetamol	300	40	100	120	2

Code	Medicine	Stock	Daily Requirement	Minimum Stock	Expiry Days	Essentiality
MED03	Insulin	60	12	50	30	3
MED04	Amoxicillin	200	25	80	20	3
MED05	Antacid	250	18	60	15	1
MED06	Cetirizine	180	15	50	90	1

These values are the initial conditions provided in the assignment.

2.3 Stock-Related Constraints

Maintaining a small stock minimizes wastage but increases the risk of critical shortages, particularly for essential medicines that require immediate attention. The system must prevent stock from dropping below zero during issuance. The program therefore calculates:

$$\text{Estimated Stock Coverage} = \text{Available Stock} / \text{Average Daily Requirement}$$

The result represents the approximate number of days for which the current stock can satisfy the average daily demand. If the daily requirement is zero, a safe fallback value of 9999.0 is used to indicate infinite coverage and prevent division-by-zero errors.

2.4 Expiry-Related Constraints

Excessive stock can lead to financial wastage if the medicines expire before they can be consumed. The system must track remaining shelf life against average daily requirements to flag projected wastage.

2.5 Conflicting Requirements

The system has to balance several conflicting requirements:

- Replenishing low-stock medicine versus avoiding unnecessary replenishment.
- Addressing medicines with low stock while also considering if the current stock is nearing expiry (Combined Shortage and Expiry).
- Simply recommending a large bulk order is inappropriate in a combined risk scenario; the system must intelligently identify this conflict to allow for smaller, fresher batch orders.

3. Proposed Solution

3.1 Stock Coverage Calculation

The program calculates stock coverage using:

$$\text{Coverage Days} = \text{Available Stock} / \text{Average Daily Requirement}$$

For example, for Oral Saline:

$$\text{Coverage} = 110 / 35 = 3.14 \text{ days (after issuing 30 units)}$$

Therefore, the updated Oral Saline stock can approximately satisfy the average requirement for 3.14 days.

3.2 Stock Condition Decision Rules

The following logical rules are used in a top-down cascading decision tree:

Condition	Stock Condition
Essentiality is 3 AND Stock Coverage < 7.0 days	Critical Condition
Available Stock < Minimum AND (Days to Expiry <= 30 OR Stock Coverage > Days to Expiry)	Combined Shortage & Expiry
Available Stock < Minimum AND Essentiality >= 2	Urgent Reorder
Days to Expiry <= 30 OR Stock Coverage > Days to Expiry	Expiry Attention
Available Stock < Minimum	Reorder Required
None of the above apply	Sufficient Stock

These rules distinguish between shortage risk, expiry risk, and combined shortage and expiry risk as required by the assignment.

3.3 Management Priority Method

A points-based priority system (maximum 145 points) is used. The final priority is calculated automatically from five factors.

Essentiality Points

- Essentiality Level * 20 points

Shortage Points

- +30 points if Stock < Minimum

Coverage Points

- +20 points if < 7.0 days
- +10 points if < 14.0 days

Expiry Points

- +20 points if <= 15 days
- +10 points if <= 30 days

Wastage Points

- +15 points if Coverage > Days to Expiry

Therefore:

Management Priority = Essentiality Points + Shortage Points + Coverage Points + Expiry Points + Wastage Points

A higher score means greater management attention is required. Average Daily Requirement is not directly scored, as its impact is inherently represented within the Stock Coverage factor, preventing double-counting.

3.4 Equal-Priority Cases

If two medicines achieve the same Priority Score, their original record order is retained via tie-breakers (Essentiality, Coverage, Expiry, Code). This provides a simple and deterministic way of resolving equal-priority cases without performing unnecessary additional calculations.

3.5 Stock and Expiry Conflict

The program does not treat every low-stock medicine in exactly the same way. A major conflict occurs when a medicine requires replenishment because its stock is low, but the current stock is also nearing expiry. The system intelligently identifies this conflict to allow for smaller, fresher batch orders. This helps distinguish:

- Shortage risk
- Expiry risk

- Combined shortage and expiry risk This is consistent with the assignment requirement.

4. Programme Design

4.1 Data Organisation

A struct Medicine is used to encapsulate all related data for a single medicine. The programme maintains an array of these structs.

The struct fields represent:

Field	Information
medCode	Medicine Code
ame	Medicine Name
□vailableStock	Available Stock
□vgDailyReq	Average Daily Requirement
minStockLevel	Minimum Stock
daysToExpiry	Days Remaining to Expiry
essentiality	Essentiality Level
stockCoverage	Calculated Stock Coverage
stockCondition	Calculated Stock Condition
priorityScore	Calculated Priority Score

This organisation ensures data remains associated during sorting and updates and allows the program to process multiple medicines using loops and functions.

4.2 Major Functions

The major functions used in the program are:

calculateDerivedFields()

Computes coverage, condition, and priority score.

searchMedicine()

Performs a case-insensitive linear search for a medicine using its identifying information.

```
C:\Users\LENOVO\AppData\Local\Microsoft\Windows\Terminal\...
=====
Medicine Stock Management System
=====
1. Search Medicine
2. Receive Stock (Add)
3. Issue Medicine (Subtract)
4. View Final Stock Report
5. Save Final Report to File
6. Exit
=====
Enter choice: 1
Enter Medicine Code to search: MED01

--- Medicine Details ---
Code: MED01
Name: Oral Saline
Available Stock: 120
Avg Daily Req: 35
Minimum Stock Level: 80
Days to Expiry: 45
Essentiality Level: 3
Estimated Coverage: 3.43 days
Stock Condition: Critical Condition
Priority Score: 80
=====

Medicine Stock Management System
=====
1. Search Medicine
2. Receive Stock (Add)
3. Issue Medicine (Subtract)
4. View Final Stock Report
5. Save Final Report to File
6. Exit
=====
Enter choice: |
```

Figure: Search Medicine Function Output

updateStock()

Handles addition of new stock and issuance of stock to patients with validation.

```
C:\Users\LENOVO\AppData\Local\Microsoft\Windows\Terminal\...
=====
Medicine Stock Management System
=====
1. Search Medicine
2. Receive Stock (Add)
3. Issue Medicine (Subtract)
4. View Final Stock Report
5. Save Final Report to File
6. Exit
=====
Enter choice: 2
Enter Medicine Code: MED01
Enter quantity to receive: 20
Successfully received 20 units of MED01.

Medicine Stock Management System
=====
1. Search Medicine
2. Receive Stock (Add)
3. Issue Medicine (Subtract)
4. View Final Stock Report
5. Save Final Report to File
6. Exit
=====
Enter choice: |
```

Figure: Receive Stock Function Output

```

C:\Users\LENOVO\AppData\Local\Microsoft\Windows\Terminal\...
=====
Medicine Stock Management System
=====
1. Search Medicine
2. Receive Stock (Add)
3. Issue Medicine (Subtract)
4. View Final Stock Report
5. Save Final Report to File
6. Exit
=====
Enter choice: 3
Enter Medicine Code: MED01
Enter quantity to issue: 30
Successfully issued 30 units of MED01.
=====
Medicine Stock Management System
=====
1. Search Medicine
2. Receive Stock (Add)
3. Issue Medicine (Subtract)
4. View Final Stock Report
5. Save Final Report to File
6. Exit
=====
Enter choice: |

```

Figure: Issue Medicine Function Output

sortMedicines()

Implements Bubble Sort based on priority to sort medicine records.

```

"medicine_stock_management.c" - Code::Blocks 23.03
File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DocyBlocks Settings Help
- global - sortMedicines(struct Medicine meds[], int count): void
Management
View: All local s
Search:
352
353
354 void generateReport(struct Medicine meds[], int count, int toFile) {
355     FILE *fp = NULL;
356     if (toFile) {
357         fp = fopen("stock_report.txt", "w");
358         if (fp == NULL) {
359             printf("\nError: Could not open file for writing.\n");
360             return;
361         }
362     }
363     #define OUT(...) do { \
364         if (toFile) fprintf(fp, __VA_ARGS__); \
365         else printf(__VA_ARGS__); \
366     } while(0)
367     OUT("\n=====");
368     OUT("%-8s | %-15s | %-6s | %-8s | %-10s | %-5s | %-28s | %-8s\n",
369         "Code", "Name", "Stock", "Qty(d)", "Exp", "Eas", "Condition", "Priority");
370     OUT("=====");
371     for (int i = 0; i < count; i++) {
372         OUT("%-8s | %-15s | %-6s | %-8.2f | %-10d | %-5d | %-28s | %-8d\n",
373             meds[i].code,
374             meds[i].name,
375             meds[i].availableStock,
376             meds[i].stockCoverage,
377             meds[i].daysToExpiry,
378             meds[i].essentiality,
379             getConditionString(meds[i].stockCondition),
380             meds[i].priorityScore);
381     }
382     OUT("=====");
383     if (toFile) {
384         fclose(fp);
385     }
386 }
387 void displaySummary(struct Medicine meds[], int count, int toFile) {
388
389
390
391

```

Figure: sortMedicines() Function Implementation

generateReport()

Displays the final summary of the medicine stock condition to the console and stores the final report in a text file.

```

394 if (toFile) {
395     fp = fopen("stock_report.txt", "a");
396     if (fp == NULL) return;
397 }
398
399 int reorderCount = 0;
400 int urgentCount = 0;
401 int expiryConcernCount = 0;
402
403 for (int i = 0; i < count; i++) {
404     if (meds[i].stockCondition == REORDER_REQUIRED ||
405         meds[i].stockCondition == URGENT_REORDER ||
406         meds[i].stockCondition == COMBINED_SHORTAGE_EXPIRY ||
407         meds[i].stockCondition == CRITICAL_CONDITION) {
408         reorderCount++;
409     }
410
411     if (meds[i].stockCondition == URGENT_REORDER ||
412         meds[i].stockCondition == CRITICAL_CONDITION) {
413         urgentCount++;
414     }
415
416     if (meds[i].stockCondition == EXPIRY_ATTENTION ||
417         meds[i].stockCondition == COMBINED_SHORTAGE_EXPIRY) {
418         expiryConcernCount++;
419     }
420 }
421
422 #define OUTS(...) do { \
423     if (toFile) fprintf(fp, __VA_ARGS__); \
424     else printf(__VA_ARGS__); \
425 } while(0)
426
427 OUTS("\n--- Summary ---\n");
428 OUTS("Total medicines analysed:      %d\n", count);
429 OUTS("Number requiring reorder:      %d\n", reorderCount);
430 OUTS("Number requiring urgent attention: %d\n", urgentCount);
431 OUTS("Number having expiry concern:    %d\n", expiryConcernCount);
432
433 if (count > 0) {
434     OUTS("Medicine requiring highest priority: %s (%s)\n", meds[0].code, meds[0].name);

```

Figure: generateReport() Function Implementation

The use of functions makes repeated operations easier to manage and keeps the program organised.

5. Optimisation Considerations

The program follows several optimisation principles required by the assignment.

5.1 Reuse of Calculated Results

Stock coverage and priority values are calculated and stored rather than repeatedly calculating the same values unnecessarily.

5.2 Use of Functions

Repeated operations such as coverage calculation, priority calculation, searching, updating and reporting are separated into functions. This reduces code repetition and makes the program easier to maintain.

5.3 Efficient Searching

The program searches the medicine records directly and stops once the required medicine is found.

5.4 Sorting

A simple sorting technique is used to arrange medicines according to management priority. All information belonging to a medicine is moved together so that records do

not become mismatched.

5.5 Storage

The program uses structured arrays to store the required information without creating unnecessary separate variables for every medicine.

6. Implementation and Results

The program is implemented using the C programming language and applies structured-programming concepts including:

- Variables and data types
- Arithmetic operators
- Relational and logical operators
- Conditional statements
- Loops
- Arrays (Structures)
- Strings
- User-defined functions
- Pass-by-reference pointers
- Searching and Sorting
- File input/output

These techniques are directly relevant to the programming knowledge expected by the assignment.

6.1 Initial Stock Coverage Results

Using the supplied dataset:

Medicine	Calculation	Coverage
Oral Saline	120 / 35	3.43 days
Paracetamol	300 / 40	7.50 days
Insulin	60 / 12	5.00 days
Amoxicillin	200 / 25	8.00 days
Antacid	250 / 18	13.89 days

Medicine	Calculation	Coverage
Cetirizine	180 / 15	12.00 days

6.2 Initial Stock Condition Results

Medicine	Stock Condition
Oral Saline	Critical Condition
Paracetamol	Sufficient Stock
Insulin	Critical Condition
Amoxicillin	Expiry Attention
Antacid	Expiry Attention
Cetirizine	Sufficient Stock

Note: The condition is produced by the program's decision rules and is not hard-coded.

6.3 Initial Priority Results

Medicine	Priority Points
Insulin	90
Oral Saline	80
Amoxicillin	80
Paracetamol	50
Antacid	50
Cetirizine	30

Therefore, Insulin receives the highest management priority because it has high essentiality and a critically short coverage.

6.4 Final Summary

Based on the initial dataset and the selected decision rules:

- Total medicines analysed: 6
- Medicines requiring reorder: 2
- Medicines requiring urgent attention: 2
- Medicines having expiry concern: 2
- Highest-priority medicine: Insulin

The final report includes medicine code, name, available stock, estimated stock coverage, expiry period, essentiality level, stock condition and management priority, as required.

```

1 #include <stdio.h>
2 #include <string.h>
3 #include <stdlib.h>
4 #include <ctype.h>
5
6 #define MAX_MEDICINES 100
7
8 // Enums for stock condition (lower value = higher severity)
9 enum StockCondition {
10     CRITICAL_CONDITION = 1,
11     COMBINED_SHORTAGE_EXPIRY = 2,
12     URGENT_REORDER = 3,
13     EXPIRY_ATTENTION = 4,
14     REORDER_REQUIRED = 5,
15     SUFFICIENT_STOCK = 6
16 };
17
18 // Struct definition
19 struct Medicine {
20     char code[10];
21     char name[50];
22     int availableStock;
23     int avgDailyReq;
24     int minStockLevel;
25     int daysToExpiry;
26     int essentiality;
27
28     double stockCoverage;
29     enum StockCondition stockCondition;
30     int priorityScore;
31 };
32
33 // Function prototypes
34 void initializeData(struct Medicine meds[], int *count);

```

```

34 void initializeData(struct Medicine meds[], int *count){
35 void calculateDerivedFields(struct Medicine *med);
36 void processAllMedicines(struct Medicine meds[], int count);
37 void displayMenu();
38 int searchMedicine(struct Medicine meds[], int count, const char * code);
39 void displaySingleMedicine(struct Medicine med);
40 void updateStock(struct Medicine meds[], int count, int issue);
41 void sortMedicines(struct Medicine meds[], int count);
42 void generateReport(struct Medicine meds[], int count, int toFile);
43 void displaySummary(struct Medicine meds[], int count, int toFile);
44 const char * getConditionString(enum StockCondition cond);
45 void clearInputBuffer();
46
47 int main() {
48     struct Medicine meds[MAX_MEDICINES];
49     int count = 0;
50     int choice = 0;
51
52     initializeData(meds, &count);
53     processAllMedicines(meds, count);
54     sortMedicines(meds, count);
55
56     while (1) {
57         displayMenu();
58         if (scanf("%d", &choice) != 1) {
59             if (feof(stdin)) {
60                 break;
61             }
62             clearInputBuffer();
63             printf("Invalid input. Please enter a number.\n");
64             continue;
65         }
66         clearInputBuffer();
67
68         switch (choice) {
69             case 1: // Search
70                 char searchCode[20];
71                 printf("Enter Medicine Code to search: ");
72                 if (fgets(searchCode, sizeof(searchCode), stdin)) {
73                     searchCode[strcspn(searchCode, "\n")] = '\0';
74                     int index = searchMedicine(meds, count, searchCode);

```

```

73 searchCode[strcspn(searchCode, "\n")] = '\0';
74 int index = searchMedicine(meds, count, searchCode);
75 if (index != -1) {
76     displaySingleMedicine(meds[index]);
77 } else {
78     printf("Error: Medicine code '%s' not found in the system.\n", searchCode);
79 }
80 break;
81
82 case 2: // Receive
83     updateStock(meds, count, 0);
84     sortMedicines(meds, count);
85     break;
86 case 3: // Issue
87     updateStock(meds, count, 1);
88     sortMedicines(meds, count);
89     break;
90 case 4: // View Report
91     generateReport(meds, count, 0);
92     displaySummary(meds, count, 0);
93     break;
94 case 5: // Save Report
95     generateReport(meds, count, 1);
96     displaySummary(meds, count, 1);
97     printf("Report successfully saved to stock_report.txt\n");
98     break;
99 case 6: // Exit
100     printf("Exiting program...\n");
101     return 0;
102 default:
103     printf("Invalid choice. Please try again.\n");
104 }
105 }
106
107 return 0;
108 }
109
110 void clearInputBuffer() {
111     int c;
112     while ((c = getchar()) != '\n' && c != EOF) {

```

```

115 // Initialize Data (struct Medicine meds[] , int *count) {
116 // Initial 6 default entries
117 struct Medicine initial[] = {
118     ("MED01", "Oral Saline", 120, 35, 80, 45, 3, 0.0, SUFFICIENT_STOCK, 0),
119     ("MED02", "Paracetamol", 200, 40, 100, 120, 2, 0.0, SUFFICIENT_STOCK, 0),
120     ("MED03", "Insulin", 80, 12, 50, 30, 3, 0.0, SUFFICIENT_STOCK, 0),
121     ("MED04", "Amoxicillin", 200, 25, 80, 20, 3, 0.0, SUFFICIENT_STOCK, 0),
122     ("MED05", "Aspirin", 250, 18, 60, 15, 1, 0.0, SUFFICIENT_STOCK, 0),
123     ("MED06", "Ceftriaxone", 180, 15, 50, 90, 1, 0.0, SUFFICIENT_STOCK, 0)
124 };
125
126 *count = 6;
127 for (int i = 0; i < *count; i++) {
128     meds[i] = initial[i];
129 }
130
131 }
132
133 void calculateDerivedFields(struct Medicine *med) {
134     // 1. Calculate Stock Coverage
135     if (med->availableReq > 0) {
136         med->stockCoverage = (double)med->availableStock / med->avgDailyReq;
137     } else {
138         med->stockCoverage = 9999.0;
139     }
140
141     // 2. Calculate Stock Condition (Top-down evaluation)
142     if (med->essentiality == 3 && med->stockCoverage < 7.0) {
143         med->stockCondition = CRITICAL_CONDITION;
144     }
145     else if (med->availableStock < med->minStockLevel &&
146             (med->daysToExpiry <= 20 || med->stockCoverage > med->daysToExpiry)) {
147         med->stockCondition = COMBINED_SHORTAGE_EXPIRY;
148     }
149     else if (med->availableStock < med->minStockLevel && med->essentiality == 2) {
150         med->stockCondition = URGENT_REORDER;
151     }
152     else if (med->daysToExpiry <= 30 || med->stockCoverage > med->daysToExpiry) {
153         med->stockCondition = EXPIRY_ATTENTION;
154     }
155     else if (med->availableStock < med->minStockLevel) {
156         med->stockCondition = REORDER_REQUIRED;
157     }
158     else {
159         med->stockCondition = SUFFICIENT_STOCK;
160     }
161
162     // 3. Calculate Management Priority Score
163     int score = 0;
164
165     // Essentiality
166     score += med->essentiality * 20;
167
168     // Shortage
169     if (med->availableStock < med->minStockLevel) {
170         score += 30;
171     }
172
173     // Coverage
174     if (med->stockCoverage < 7.0) {
175         score += 20;
176     } else if (med->stockCoverage < 14.0) {
177         score += 10;
178     }
179
180     // Expiry
181     if (med->daysToExpiry <= 15) {
182         score += 20;
183     } else if (med->daysToExpiry <= 30) {
184         score += 10;
185     }
186
187     // Wastage
188     if (med->stockCoverage > med->daysToExpiry) {
189         score += 15;
190     }
191
192     med->priorityScore = score;
193 }
194

```

```

154 }
155
156 else if (med->availableStock < med->minStockLevel) {
157     med->stockCondition = REORDER_REQUIRED;
158 }
159
160 else {
161     med->stockCondition = SUFFICIENT_STOCK;
162 }
163
164 // 3. Calculate Management Priority Score
165 int score = 0;
166
167 // Essentiality
168 score += med->essentiality * 20;
169
170 // Shortage
171 if (med->availableStock < med->minStockLevel) {
172     score += 30;
173 }
174
175 // Coverage
176 if (med->stockCoverage < 7.0) {
177     score += 20;
178 } else if (med->stockCoverage < 14.0) {
179     score += 10;
180 }
181
182 // Expiry
183 if (med->daysToExpiry <= 15) {
184     score += 20;
185 } else if (med->daysToExpiry <= 30) {
186     score += 10;
187 }
188
189 // Wastage
190 if (med->stockCoverage > med->daysToExpiry) {
191     score += 15;
192 }
193
194 med->priorityScore = score;
195

```

```

193 }
194
195 void processAllMedicines(struct Medicine meds[], int count) {
196     for (int i = 0; i < count; i++) {
197         calculateDerivedFields(&meds[i]);
198     }
199 }
200
201 void displayMenu() {
202     printf("\n=====n");
203     printf(" Medicine Stock Management System\n");
204     printf("=====n");
205     printf("1. Search Medicine\n");
206     printf("2. Receive Stock (Add)\n");
207     printf("3. Issue Medicine (Subtract)\n");
208     printf("4. View Final Stock Report\n");
209     printf("5. Save Final Report to File\n");
210     printf("6. Exit\n");
211     printf("-----n");
212     printf("Enter choice: ");
213 }
214
215 int searchMedicine(struct Medicine meds[], int count, const char* code) {
216     for (int i = 0; i < count; i++) {
217         // Case insensitive compare (simple implementation)
218         const char *p1 = meds[i].code;
219         const char *p2 = code;
220         int match = 1;
221         while (*p1 != '\0' || *p2 != '\0') {
222             if (tolower((unsigned char)*p1) != tolower((unsigned char)*p2)) {
223                 match = 0;
224                 break;
225             }
226             p1++;
227             p2++;
228         }
229         if (match) {
230             return i;
231         }
232     }
233     return -1;
234 }

```

```

"medicine_stock_management.c - CodeBlocks 25.03
File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoryBlocks Settings Help
<global>
searchMedicine(struct Medicine meds[], int count, const char* code): int
Management
Symbols Resources
View: All local s
Search:
235
236 const char* getConditionString(enum StockCondition cond) {
237     switch (cond)
238     {
239         case CRITICAL_CONDITION: return "Critical Condition";
240         case COMBINED_SHORTAGE_EXPIRY: return "Combined Shortage & Expiry";
241         case URGENT_REORDER: return "Urgent Reorder";
242         case EXPIRY_ATTENTION: return "Expiry Attention";
243         case REORDER_REQUIRED: return "Reorder Required";
244         case SUFFICIENT_STOCK: return "Sufficient Stock";
245         default: return "Unknown";
246     }
247 }
248
249 void displaySingleMedicine(struct Medicine med) {
250     printf("\n--- Medicine Details ---\n");
251     printf("Code: %s\n", med.code);
252     printf("Name: %s\n", med.name);
253     printf("Available Stock: %d\n", med.availableStock);
254     printf("Avg Daily Req: %d\n", med.avgDailyReq);
255     printf("Minimum Stock Level: %d\n", med.minStockLevel);
256     printf("Days to Expiry: %d\n", med.daysToExpiry);
257     printf("Essentiality Level: %d\n", med.essentiality);
258     printf("Estimated Coverage: %.2f days\n", med.stockCoverage);
259     printf("Stock Condition: %s\n", getConditionString(med.stockCondition));
260     printf("Priority Score: %d\n", med.priorityScore);
261     printf("-----\n");
262 }
263
264 void updateStock(struct Medicine meds[], int count, int isIssue) {
265     char searchCode[20];
266     int qty;
267
268     printf("\nEnter Medicine Code: ");
269     if (fgets(searchCode, sizeof(searchCode), stdin)) {
270         searchCode[strcspn(searchCode, "\n")] = '\0';
271     } else {
272         return;
273     }
274
275     int index = searchMedicine(meds, count, searchCode);
276     if (index == -1) {

```

```

"medicine_stock_management.c - CodeBlocks 25.03
File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoryBlocks Settings Help
<global>
updateStock(struct Medicine meds[], int count, int isIssue): void
Management
Symbols Resources
View: All local s
Search:
274
275 int index = searchMedicine(meds, count, searchCode);
276 if (index == -1) {
277     printf("\nError: Medicine code '%s' not found.\n", searchCode);
278     return;
279 }
280
281 if (isIssue) {
282     printf("Enter quantity to issue: ");
283 } else {
284     printf("Enter quantity to receive: ");
285 }
286
287 if (scanf("%d", &qty) != 1) {
288     clearInputBuffer();
289     printf("\nError: Invalid quantity input.\n");
290     return;
291 }
292 clearInputBuffer();
293
294 if (qty <= 0) {
295     printf("\nError: Quantity must be greater than zero.\n");
296     return;
297 }
298
299 if (isIssue) {
300     if (qty > meds[index].availableStock) {
301         printf("\nError: Issued quantity (%d) exceeds available stock (%d).\n", qty, meds[index].availableStock);
302         return;
303     }
304     meds[index].availableStock -= qty;
305     printf("\nSuccessfully issued %d units of %s.\n", qty, meds[index].code);
306 } else {
307     meds[index].availableStock += qty;
308     printf("\nSuccessfully received %d units of %s.\n", qty, meds[index].code);
309 }
310
311 // Recalculate derived fields
312 calculateDerivedFields(meds[index]);
313
314 void sortMedicines(struct Medicine meds[], int count) {

```

```

"medicine_stock_management.c - CodeBlocks 25.03
File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoryBlocks Settings Help
<global>
updateStock(struct Medicine meds[], int count, int isIssue): void
Management
Symbols Resources
View: All local s
Search:
313
314 void sortMedicines(struct Medicine meds[], int count) {
315     for (int i = 0; i < count - 1; i++) {
316         for (int j = 0; j < count - i - 1; j++) {
317             int swap = 0;
318
319             // 1. Higher Priority Score wins
320             if (meds[j].priorityScore < meds[j+1].priorityScore) {
321                 swap = 1;
322             } else if (meds[j].priorityScore == meds[j+1].priorityScore) {
323                 // Tie-breakers
324                 // 1st: Higher Essentiality Level
325                 if (meds[j].essentiality < meds[j+1].essentiality) {
326                     swap = 1;
327                 } else if (meds[j].essentiality == meds[j+1].essentiality) {
328                     // 2nd: Lower Stock Coverage
329                     if (meds[j].stockCoverage > meds[j+1].stockCoverage) {
330                         swap = 1;
331                     } else if (meds[j].stockCoverage == meds[j+1].stockCoverage) {
332                         // 3rd: Lower Days Remaining to Expiry
333                         if (meds[j].daysToExpiry > meds[j+1].daysToExpiry) {
334                             swap = 1;
335                         } else if (meds[j].daysToExpiry == meds[j+1].daysToExpiry) {
336                             // 4th: Alphabetically smaller Medicine Code
337                             if (strcmp(meds[j].code, meds[j+1].code) > 0) {
338                                 swap = 1;
339                             }
340                         }
341                     }
342                 }
343             }
344             if (swap) {
345                 struct Medicine temp = meds[j];
346                 meds[j] = meds[j+1];
347                 meds[j+1] = temp;
348             }
349         }
350     }
351 }
352
353 }

```

```

175 void generateReport(struct Medicine meds[], int count, int toFile) {
176     FILE *fp = NULL;
177     if (toFile) {
178         fp = fopen("stock_report.txt", "w");
179         if (fp == NULL) {
180             printf("\nError: Could not open file for writing.\n");
181             return;
182         }
183     }
184     #define OUT(...) do { \
185         if (toFile) fprintf(fp, __VA_ARGS__); \
186         else printf(__VA_ARGS__); \
187     } while(0)
188     OUT("\n=====");
189     OUT("%-8s | %-15s | %-6s | %-8s | %-10s | %-5s | %-28s | %-8s\n",
190         "Code", "Name", "Stock", "Exp(d)", "Eas", "Condition", "Priority");
191     OUT("=====");
192     for (int i = 0; i < count; i++) {
193         OUT("%-8s | %-15s | %-6d | %-8.2f | %-10d | %-5d | %-28s | %-8d\n",
194             meds[i].code,
195             meds[i].name,
196             meds[i].stockCover,
197             meds[i].stockCondition,
198             meds[i].daysToExpiry,
199             getConditionString(meds[i].stockCondition),
200             meds[i].priorityScore);
201     }
202     OUT("=====");
203     if (toFile) {
204         fclose(fp);
205     }
206 }
207 void displaySummary(struct Medicine meds[], int count, int toFile) {

```

```

394 if (toFile) {
395     fp = fopen("stock_report.txt", "a");
396     if (fp == NULL) return;
397 }
398
399 int reorderCount = 0;
400 int urgentCount = 0;
401 int expiryConcernCount = 0;
402
403 for (int i = 0; i < count; i++) {
404     if (meds[i].stockCondition == REORDER_REQUIRED ||
405         meds[i].stockCondition == URGENT_REORDER ||
406         meds[i].stockCondition == COMBINED_SHORTAGE_EXPIRY ||
407         meds[i].stockCondition == CRITICAL_CONDITION) {
408         reorderCount++;
409     }
410     if (meds[i].stockCondition == URGENT_REORDER ||
411         meds[i].stockCondition == CRITICAL_CONDITION) {
412         urgentCount++;
413     }
414     if (meds[i].stockCondition == EXPIRY_ATTENTION ||
415         meds[i].stockCondition == COMBINED_SHORTAGE_EXPIRY) {
416         expiryConcernCount++;
417     }
418 }
419
420 #define OUTS(...) do { \
421     if (toFile) fprintf(fp, __VA_ARGS__); \
422     else printf(__VA_ARGS__); \
423 } while(0)
424
425 OUTS("\n--- Summary ---\n");
426 OUTS("Total medicines analyzed: %d\n", count);
427 OUTS("Number requiring reorder: %d\n", reorderCount);
428 OUTS("Number requiring urgent attention: %d\n", urgentCount);
429 OUTS("Number having expiry concern: %d\n", expiryConcernCount);
430
431 if (count > 0) {
432     OUTS("Medicine requiring highest priority: %s (%s)\n", meds[0].code, meds[0].name);
433 }
434

```

```

403 for (int i = 0; i < count; i++) {
404     if (meds[i].stockCondition == REORDER_REQUIRED ||
405         meds[i].stockCondition == URGENT_REORDER ||
406         meds[i].stockCondition == COMBINED_SHORTAGE_EXPIRY ||
407         meds[i].stockCondition == CRITICAL_CONDITION) {
408         reorderCount++;
409     }
410     if (meds[i].stockCondition == URGENT_REORDER ||
411         meds[i].stockCondition == CRITICAL_CONDITION) {
412         urgentCount++;
413     }
414     if (meds[i].stockCondition == EXPIRY_ATTENTION ||
415         meds[i].stockCondition == COMBINED_SHORTAGE_EXPIRY) {
416         expiryConcernCount++;
417     }
418 }
419
420 #define OUTS(...) do { \
421     if (toFile) fprintf(fp, __VA_ARGS__); \
422     else printf(__VA_ARGS__); \
423 } while(0)
424
425 OUTS("\n--- Summary ---\n");
426 OUTS("Total medicines analyzed: %d\n", count);
427 OUTS("Number requiring reorder: %d\n", reorderCount);
428 OUTS("Number requiring urgent attention: %d\n", urgentCount);
429 OUTS("Number having expiry concern: %d\n", expiryConcernCount);
430
431 if (count > 0) {
432     OUTS("Medicine requiring highest priority: %s (%s)\n", meds[0].code, meds[0].name);
433 }
434 OUTS("=====");
435
436 if (toFile) {
437     fclose(fp);
438 }
439
440
441
442

```

Figure: Input / Source Code

```

=====
Medicine Stock Management System
=====
1. Search Medicine
2. Receive Stock (Add)
3. Issue Medicine (Subtract)
4. View Final Stock Report
5. Save Final Report to File
6. Exit
=====
Enter choice: 4
=====
Code | Name | Stock | Cov(d) | Exp(d) | Ess | Condition | Priority
=====
MED03 | Insulin | 60 | 5.00 | 30 | 3 | Critical Condition | 90
MED01 | Oral Saline | 120 | 3.43 | 45 | 3 | Critical Condition | 80
MED04 | Amoxicillin | 200 | 8.00 | 20 | 3 | Expiry Attention | 80
MED02 | Paracetamol | 300 | 7.50 | 120 | 2 | Sufficient Stock | 50
MED05 | Antacid | 250 | 13.89 | 15 | 1 | Expiry Attention | 50
MED06 | Cetirizine | 180 | 12.00 | 90 | 1 | Sufficient Stock | 30
=====

--- Summary ---
Total medicines analysed: 6
Number requiring reorder: 2
Number requiring urgent attention: 2
Number having expiry concern: 2
Medicine requiring highest priority: MED03 (Insulin)
=====

Medicine Stock Management System
=====
1. Search Medicine
2. Receive Stock (Add)
3. Issue Medicine (Subtract)
4. View Final Stock Report
5. Save Final Report to File
6. Exit
=====
Enter choice: |

```

Figure: Program Output

7. Testing and Validation

The assignment requires at least eight specified test cases and one additional independently designed test case. Each test case must contain the condition, expected result, actual result and Pass/Fail status.

TC-01: Initial Stock Report

Test Condition: Evaluate all 6 medicines correctly.

Expected Result: Output confirmed correctly calculated coverage, conditions (e.g. MED03 Critical Priority 90), and summary statistics.

Actual Result: Output confirmed correctly calculated coverage, conditions (e.g. MED03 Critical Priority 90, MED01 Critical Priority 80), and summary statistics showing 2 Reorder, 2 Urgent, 2 Expiry. Highest Priority: MED03.

Status: PASS

TC-02: Search Existing Medicine

Test Condition: Find medicine matching exactly "MED01".

Expected Result: Oral Saline found with correct details.

Actual Result: Oral Saline found with correct details.

Status: PASS

TC-03: Case-Insensitive Search

Test Condition: Find medicine using lowercase "med01".

Expected Result: MED01 / Oral Saline successfully found.

Actual Result: MED01 / Oral Saline successfully found.

Status: PASS

TC-04: Medicine Not Found

Test Condition: Search for non-existent "MED99".

Expected Result: Error message displayed.

Actual Result: Produced error: "Error: Medicine code 'MED99' not found in the system."

Status: PASS

TC-05: Receive Stock

Test Condition: Add 20 units to MED01 (Initial: 120).

Expected Result: Stock changed from 120 to 140.

Actual Result: "Successfully received 20 units of MED01." Stock changed from 120 to 140.

Status: PASS

TC-06: Issue Medicine

Test Condition: Subtract 30 units from MED01 (Current: 140).

Expected Result: Stock changed from 140 to 110.

Actual Result: "Successfully issued 30 units of MED01." Stock changed from 140 to 110.

Status: PASS

TC-07: Over-Issue Validation

Test Condition: Attempt to issue 200 units from available 110.

Expected Result: Error message displayed and stock protected.

Actual Result: "Error: Issued quantity (200) exceeds available stock (110)." Stock protected from negative balance.

Status: PASS

TC-08: Dynamic Recalculation

Test Condition: Coverage must update after TC-05 and TC-06.

Expected Result: Coverage properly recalculated to 3.14 days.

Actual Result: Runtime report showed: MED01 | Oral Saline | 110 | 3.14 | 45 | 3 | Critical Condition | 80.

Status: PASS

TC-09: Zero Average Daily Demand

Test Condition: Entering a medicine with avgDailyReq = 0.

Expected Result: Coverage = 9999.00 without division by zero.

Actual Result: Zero demand correctly produced Coverage = 9999.00 without division by zero. Output: MED01 | Oral Saline | 120 | 9999.00 | 45 | 3 | Expiry Attention | 75.

Status: PASS

8. Limitations

The developed program has some limitations:

1. Bubble sort is used for simplicity and stability, which is highly efficient for small arrays but would require optimization (e.g., Quicksort) if scaling to tens of thousands of medicines.
2. The program saves reports to a file, but currently resets the medicine inventory to the initial 6 records upon restart. A full database system would be required for persistent inventory tracking across reboots.

Despite these limitations, the solution satisfies the main functional requirements of the assignment and demonstrates the required structured-programming techniques.

9. Conclusion

The Medicine Stock and Expiry Management program provides a structured method for analysing medicine inventory in a community health center.

The program calculates estimated stock coverage, evaluates stock and expiry conditions, considers medicine essentiality, assigns management priority, supports medicine searching and stock updates, sorts medicines according to priority, and generates a final report.

The solution also handles important situations such as shortage, expiry risk and combined shortage-and-expiry risk. Stock cannot become negative during medicine issuing. The program uses arrays, strings, loops, conditional statements, functions, searching, sorting and file operations, which are among the programming techniques expected in the CSC 1383 assignment.

Based on the initial dataset, Insulin receives the highest management priority because of its high essentiality and expiry concern. Therefore, the program provides useful information that can help health-center staff decide which medicines should receive management attention first.

Overall, the developed system provides a simple, readable and computationally efficient solution to the medicine stock and expiry management problem.

10. References

[1] Abdullah Mohammad Sakib, "Assignment on Medicine Stock and Expiry Management," CSC 1383 Structured Programming, Summer 2026.